

```
"""
```

```
draft_sender.py
```

```
=====
```

```
Creates a REVIEWABLE DRAFT in Gmail with the LOI attached. It never sends.
```

```
There is no send call anywhere in this module. `users().messages().send` is not
imported, not wrapped, not commented out. If you later want a send path, add it
deliberately -- do not let it creep in.
```

```
Outreach hygiene
```

```
-----
```

```
This is individualised business-to-business correspondence to a listing agent
whose job is to receive proposals, and every message passes a human before it
leaves. Keep it that way:
```

- * Do not mass-generate drafts you have not actually read.
- * Do not re-contact the same agent about the same property on a timer.
- * Honour any request to stop.
- * Never state a licensing status, an endorsement, or a capability you cannot document. This message goes to a licensed professional who may forward it to a property owner and to their own broker.

```
"""
```

```
from __future__ import annotations
```

```
import base64
```

```
import logging
```

```
import mimetypes
```

```
from email.message import EmailMessage
```

```
from pathlib import Path
```

```
from typing import Any, Optional
```

```
import config
```

```
from models import AFHAssessment, DealEconomics, Listing
```

```
log = logging.getLogger(__name__)
```

```
SUBJECT_TEMPLATE = (
```

```
    "Corporate Lease Proposal & LOI: {address} - Guaranteed Long-Term Tenancy"
)
```

```
# -----
```

```
# Body copy
```

```
# -----
```

```
def build_email_body(
    listing: Listing,
    economics: Optional[DealEconomics] = None,
    assessment: Optional[AFHAssessment] = None,
    *,
    company: config.CompanyProfile = config.COMPANY,
    lease: config.LeaseTerms = config.LEASE,
) -> str:
    """Plain-text broker-to-operator pitch. Deliberately claim-light."""
    agent_first = (listing.agent.name or "").split()[0] if listing.agent.name else
    greeting = f"Hi {agent_first}," if agent_first else "Hello,"

    p2 = economics.proposed_phase2_rent if economics else (
        assessment.suggested_phase2_full_rent if assessment else None)
    p1 = economics.proposed_phase1_rent if economics else (
        assessment.suggested_phase1_holding_rent if assessment else None)

    rent_line = ""
    if p1 and p2:
        rent_line = (
            f"- A two-phase rent structure: a reduced holding rent of "
            f"${p1:,.0f}/month during the first {lease.phase1_months} months "
            f"while inspections and licensing are completed, stepping up to "
            f"${p2:,.0f}/month for the balance of the term.\n"
        )

    address_line = f"{listing.address}, {listing.city}, OR {listing.zip_code or ' '

    return f""{greeting}
```

I am writing about {address_line}.

I operate a licensed residential care home in Clackamas County and I am looking to expand into a second property. Rather than a conventional purchase offer, I would like to propose a corporate master lease with an option to purchase. For an owner who is carrying a property that has been on the market for a while, this structure often solves the problem faster than waiting for the right buyer.

What I am proposing, in short:

- A {lease.term_years_min} to {lease.term_years_max} year corporate master lease. One corporate tenant, one signature, rent paid on the first of the month regardless of the home's internal occupancy.
- {rent_line}- The home is occupied and staffed around the clock, professionally cleaned and maintained. No vacancy risk, no undetected leaks, no turnover

churn between individual tenants.

- \$1M/\$2M commercial general and professional liability coverage in place, with your seller named as an additional insured and certificates provided.
- Any accessibility work is done at my expense by licensed contractors with permits, subject to the owner's written approval of plans.
- An option to purchase during the term, so the owner keeps a defined exit.
- Your commission is preserved on the lease transaction and again if the purchase option is exercised, per your listing agreement.

I have attached a two-page letter of intent with the full terms. It is non-binding and meant as a starting point for a conversation, not a take-it-or-leave-it offer. Every number in it is open to discussion.

The proposal is contingent on a {lease.due_diligence_days}-day period to confirm zoning, any CC&R restrictions, and physical suitability with the county and state. I would want to walk the property with you before going further, and I am happy to speak with the owner directly if that is easier.

Would you have twenty minutes this week or next?

Thank you for your time.

Best,

{company.signature_block}

--

This letter of intent is non-binding and does not create any lease or obligation. Any agreement would require a definitive written lease executed by both parties.

""

```
# -----  
# Draft creation  
# -----
```

```
def build_mime_message(  
    to_address: str,  
    subject: str,  
    body: str,  
    attachment_path: Optional[Path] = None,  
    *,  
    from_address: Optional[str] = None,  
    cc: Optional[str] = None,  
) -> EmailMessage:  
    msg = EmailMessage()  
    msg["To"] = to_address
```

```

msg["Subject"] = subject
if from_address:
    msg["From"] = from_address
if cc:
    msg["Cc"] = cc
msg.set_content(body)

if attachment_path and Path(attachment_path).exists():
    path = Path(attachment_path)
    ctype, _ = mimetypes.guess_type(path.name)
    maintype, subtype = (ctype or "application/pdf").split("/", 1)
    msg.add_attachment(
        path.read_bytes(),
        maintype=maintype,
        subtype=subtype,
        filename=path.name,
    )
elif attachment_path:
    log.warning("Attachment not found, drafting without it: %s", attachment_p

return msg

def create_draft(
    service,
    listing: Listing,
    loi_path: Optional[Path] = None,
    economics: Optional[DealEconomics] = None,
    assessment: Optional[AFHAssessment] = None,
    *,
    company: config.CompanyProfile = config.COMPANY,
    dry_run: bool = True,
) -> Optional[str]:
    """Create a Gmail draft. Returns the draft id, or None.

    Raises ValueError when there is no agent email -- the caller should have
    checked `listing.agent.is_contactable` first.
    """
    if not listing.agent.is_contactable:
        raise ValueError(
            f"No listing-agent email for {listing.address}. "
            "Supply one before drafting."
        )

    subject = SUBJECT_TEMPLATE.format(
        address=f"{listing.address}, {listing.city}"
    )

```

```

body = build_email_body(listing, economics, assessment, company=company)
message = build_mime_message(
    to_address=listing.agent.email, # type: ignore[arg-type]
    subject=subject,
    body=body,
    attachment_path=loi_path,
    from_address=company.email or None,
)

if dry_run:
    log.info("[DRY RUN] Would draft to %s | %s", listing.agent.email, subject)
    preview = config.OUTPUT_DIR / f"DRAFT_{listing.slug}.txt"
    preview.write_text(
        f"To: {listing.agent.email}\nSubject: {subject}\n"
        f"Attachment: {loi_path.name if loi_path else '(none)'}\n"
        f"{'-' * 70}\n{body}",
        encoding="utf-8",
    )
    log.info("Draft preview written to %s", preview)
    return None

from googleapiclient.errors import HttpError

raw = base64.urlsafe_b64encode(message.as_bytes()).decode()
try:
    draft = (
        service.users()
        .drafts()
        .create(userId="me", body={"message": {"raw": raw}})
        .execute()
    )
except HttpError as exc:
    log.error("Gmail draft creation failed for %s: %s", listing.address, exc)
    return None

draft_id = draft.get("id")
log.info("Created Gmail DRAFT %s for %s (NOT sent)", draft_id, listing.address)
return draft_id

def draft_summary(listing: Listing, draft_id: Optional[str]) -> str:
    if draft_id:
        return (f"Draft {draft_id} is waiting in Gmail for {listing.address}. "
                "Open it, read it, edit it, then send it yourself.")
    return f"No draft created for {listing.address}."

```

```
# Explicit non-capability marker.
def send(*_args: Any, **_kwargs: Any) -> None: # noqa: D401
    """Not implemented on purpose. This pipeline does not send email."""
    raise NotImplementedError(
        "This pipeline creates drafts only. Review and send from Gmail."
    )
```